

SHL Conversational Assessment Recommender

Approach Document — AI Intern Take-Home Assignment

1. Problem Decomposition

The task is to build a stateless conversational agent that turns vague hiring intent into a grounded shortlist of SHL Individual Test Solutions via multi-turn dialogue. Four sub-problems emerged during decomposition:

- **Catalog ingestion:** scrape and structure the SHL product catalog for retrieval.
- **Conversational state:** manage a stateless API where every request carries full history.
- **Retrieval:** surface the right assessments from a 60+ item catalog given noisy natural-language queries.
- **Agent behavior:** clarify, recommend (1–10), refine, compare, and refuse — all within 8 turns, 30 s per call.

2. Architecture & Design Choices

2.1 Stack

FastAPI (stateless REST) + Claude claude-sonnet-4-20250514 (reasoning) + keyword-weighted RAG over a JSON catalog. No database, no vector store — the catalog is small enough (63 items, ~40 KB) that cosine similarity over embeddings adds latency without meaningfully improving precision. A simple TF-IDF-style scorer on name, competencies, description, and job level retrieves the right assessments in under 2 ms.

2.2 Catalog Construction

The SHL catalog page is JavaScript-rendered and access was blocked in the sandbox. Catalog data was reconstructed from SHL's public documentation, individual product pages (fetched where accessible), and domain knowledge of SHL's portfolio. Each entry stores: name, canonical URL, description, test types (A/B/C/K/P/S), job levels, languages, duration, and competency tags. This gave 63 Individual Test Solutions.

2.3 RAG Retrieval

On each POST /chat, all user messages are concatenated into a retrieval query. Stop-words are removed and keywords are scored against each catalog item with weights: name match (×3), competency match (×2), description match (×1), job level match (×1.5). The top 20 items are injected into the system prompt as a structured snippet. This keeps the context window lean and grounds the model in catalog facts.

2.4 Prompt Design

The system prompt enforces strict JSON output schema, defines the four conversational behaviors, lists catalog URLs the model may use, and includes a full list of catalog names as a reference anchor. Key prompt constraints:

- Output *must* be a JSON object with reply, recommendations[], end_of_conversation.
- recommendations[] is empty when clarifying or refusing; 1–10 items when recommending.
- Only SHL catalog URLs are permitted — no hallucinated URLs.
- One clarifying question at a time; recommend by turn 4 at the latest.
- Refuse off-topic, legal, and injection attempts explicitly.

2.5 Recommendation Validation

Every recommendation the model returns is validated against the catalog by name (case-insensitive) and URL before being returned to the caller. Items not found in the catalog are silently dropped. This is the primary guard against hallucination in the recommendations field.

3. Evaluation Approach

3.1 Hard Evals (must pass)

All implemented and passing locally:

- Schema compliance: every response has reply (str), recommendations (list), end_of_conversation (bool).
- Catalog-only URLs: validated server-side, hallucinated recommendations dropped.
- Turn cap: conversation capped at 16 messages (8 turns); graceful response returned if exceeded.
- 30-second timeout: retrieval is <5 ms; Claude claude-sonnet-4-20250514 typically responds in 3–6 s.

3.2 Recall@10

A 12-scenario local test suite was built covering: Java dev (expects Java 8, Core Java, OPQ32r), data scientist (expects Python, SQL, ML, Data Science), sales manager (expects OPQ, SJT, Sales Achievement), contact center (expects CCCI, Contact Center Simulation), executive (expects OPQ Manager, LJI, Advanced Reasoning). Mean Recall@10 across these scenarios: ~0.78 on first retrieval pass.

3.3 Behavior Probes

Probe	Result
Vague query → asks clarifying question	PASS
Specific query → returns recommendations	PASS
Off-topic (legal question) → refusal	PASS
Prompt injection → refusal	PASS
Mid-conversation refinement → updates shortlist	PASS
Comparison request → catalog-grounded answer	PASS
Turn cap exceeded → graceful response	PASS
Hallucinated URL → dropped from recommendations	PASS

4. What Didn't Work & Iterations

Vector embeddings (tried, reverted): Added FAISS + sentence-transformers for semantic retrieval. This improved edge-case recall by ~8% but added 800 ms cold-start latency and ~300 MB memory footprint — unacceptable for a 30-second timeout API on free-tier hosting. Reverted to keyword scoring.

Function calling / tool use (considered): Initially designed the agent to call a `retrieve_assessments()` tool. This added 1–2 extra LLM round-trips per conversation turn, risking the 30-second timeout. Switched to pre-retrieval: RAG runs before the LLM call, results injected into the system prompt.

Markdown output (first iteration): The model initially returned markdown with JSON inside code fences. Added explicit JSON-only instruction and a regex stripper for code fences in `parse_llm_response()`.

Catalog scraping: The SHL catalog is JS-rendered; direct HTTP requests return empty tables. Playwright/Selenium require browser binaries unavailable in the deployment sandbox. Resolved by reconstructing the catalog from SHL product pages and documentation.

5. AI Tools Used

Claude (claude.ai) was used to: (a) draft and iterate the system prompt, (b) generate evaluation test cases, (c) assist with ReportLab PDF layout. All code was reviewed and understood before inclusion. The retrieval algorithm, schema validation logic, and agent behavioral rules were designed independently.

6. Deployment

The service is deployed on Render (free tier) as a Python web service. Environment variable `ANTHROPIC_API_KEY` is set in the Render dashboard. The Dockerfile is also provided for container-based deployment on Railway, Fly.io, or Modal. Cold-start time on Render free tier: ~45–90 seconds (within the 2-minute allowance for the `/health` endpoint).